# Towards Next-Generation Steganalysis: LLMs Unleash the Power of Detecting Steganography

Minhao Bai†, Jinshuai Yang†, Kaiyi Pang, Huili Wang, Yongfeng Huang



***Abstract*—Linguistic steganography provides convenient implementation to hide messages, particularly with the emergence of AI generation technology. The potential abuse of this technology raises security concerns within societies, calling for powerful linguistic steganalysis to detect carrier containing steganographic messages. Existing methods are limited to finding distribution differences between steganographic texts and normal texts from the aspect of symbolic statistics. However, the distribution differences of both kinds of texts are hard to build precisely, which heavily hurts the detection ability of the existing methods in realistic scenarios. To seek a feasible way to construct practical steganalysis in real world, this paper propose to employ human-like text processing abilities of large language models (LLMs) to realize the difference from the aspect of human perception, addition to traditional statistic aspect. Specifically, we systematically investigate the performance of LLMs in this task by modeling it as a generative paradigm, instead of traditional classification paradigm. Extensive experiment results reveal that generative LLMs exhibit significant advantages in linguistic steganalysis and demonstrate performance trends distinct from traditional approaches. Results also reveal that LLMs outperform existing baselines by a wide margin, and the domain-agnostic ability of LLMs makes it possible to train a generic steganalysis model[1].**

***Index Terms*—Linguistic Steganalysis, Large Language Models, Generative Steganalysis.**

## I. Introduction

STEGANOGRAPHY is the technology to hide messages within normal information carriers such as images [1–3], audios [4, 5], or texts [6–16]. It allows individuals to covertly transmit secret messages by embedding them within seemingly innocent content. With the most advanced generative linguistic steganography, a paragraph of text can embed a secret message of several hundred bits, making it possible to transmit complex information. It is within these inconspicuous sentences that messages are covertly conveyed.

Due to the peculiar nature of this technology, the abuse of steganography raises security concerns within societies, leading to significant anxiety among the general public [17, 18]. Unethical individuals may employ steganography to convey malicious information, such as terrorists using steganography to plan attacks [19, 20], or hackers using it to distribute virus software [21]. Therefore, the detection of steganographic carriers containing secret messages, namely steganalysis, has emerged as a widely discussed area in contemporary information security.

Understanding steganography is the foundation of steganalysis. Thanks to the widespread usage of texts and the extraordinary achievement of AI generation techiques, generative linguistic steganography has become a popular choice for research and practice. Generative linguistic steganography embeds secret messages into automatically generated texts. The advantage lies in the absence of a predefined carrier, resulting in enormous flexibility and a high embedding rate. In the text generation process, a codebook is formed using the conditional probability distribution predicted by a language model. To construct the codebook, researchers have tried various methods including heuristic rules [12], source coding based methods [13–16] and distribution preserving methods [8, 10]. After the codebook is determined, the appropriate tokens are selected based on the secret messages.

Constructing a secure codebook is critical to the imperceptibility of steganographic texts, especially in the era that large language models can build good enough conditional probability distribution. Among these codebook construction methods, heuristic methods [12] expose significant statistical distribution difference between steganographic texts and normal texts, source coding based methods [13–16] can make the deviation small, while distribution preserving methods [8, 10] can make the deviation tiny enough to near zero.

To distinguish between the steganographic texts (also called stegos) and the natural texts (also called covers), these existing linguistic steganalysis methods depend on statistical features to be aware of the distribution deviation, acting as classifiers. The earlier methods depend heavily on heuristic statistical features [22, 23], unaware of the deep features of texts. Recent approaches focus on assembling existing neural network modules to get different learning-based deep features of texts [24–32], while several recent works [33–35] choose to use pre-trained models like BERT [36] as the base module. These approaches seriously rely on the distributional consistency between training texts and target texts, which is difficult to accurately achieve.

For heuristic codebook construction methods, it is easy for existing steganalysis methods to detect steganographic texts. However, when these recent methods meet source coding-based method and distribution preserving methods, especially the latter, they can only find little distribution difference and then show weaker detection performance. As shown in Fig. 1, the detection accuracy of these existing methods is below 75%, leaving a room for further improvement.

There are two key reasons prevent these methods from

† Equal contribution.

M. Bai, J. Yang, K. Pang and H. Wang are with the Department of Electronic Engineering, Tsinghua University, Beijing, 100084, China. (e-mail: bmh22@mails.tsinghua.edu.cn)

Y. Huang is with the Zhongguancun Laboratory, Beijing, 100084, China. (email: yfhuang@tsinghua.edu.cn)

[1]Both codes and trained models are openly available in https://github.com/ba0z1/Linguistic-Steganalysis-with-LLMs

accurately perceiving the potential distribution differences in realistic world. Firstly, building precise distribution of stegos demands massive annotated text. Secondly, pinpointing the distribution difference demands oracle distribution of normal texts in target scenario. The two excessive demands force these methods relied on distribution difference to only obtain unsatisfactory performance.

Actually, there exists the contradiction between statistical distribution differences and text perception rationality for almost all the codebook construction methods [7]. This indicates that rationality of text deteriorates as distribution differences decrease, which suggests a clue for steganalyzers. Although evaluating rationality of text is not very difficult for humans, almost all the traditional steganalysis methods trained as statistical classifiers seem to lack this ability. As shown in Fig. 2, current steganalysis methods are easily fooled by low-quality stegos and irrational stegos, while these stegos can be distinguished by human. However, LLMs have shown human-like abilities in many aspects, including evaluating fluency and rationality of text [37]. For example, ChatGPT has been widely used to annotate high-quality text, replacing human annotations.

Based on the above observations, a natural thought emerged: we expect that the human-like capabilities of LLMs can help steganalyzers to achieve more accurate detection, thus overcoming the performance dilemma of current linguistic steganalysis methods. However, exploiting the capabilities of LLMs for linguistic steganalysis poses significant challenges and costs. On the one hand, activating the capability of LLMs is not easily available. Researchers have explored the potential of using ChatGPT for steganalysis tasks [38], which shows mediocre performance due to lack of focused design and training. On the other hand, due to the vast parameters of modern LLMs, direct training requires significant time and financial investment, with the risk of losing its original capabilities.

To seek a feasible way to construct practical steganalysis in realistic world, this paper systematically explores the LLMs for linguistic steganalysis. To hold human-like ability of LLMs as possible, we propose to generate readable detection results. To fully activate the domain-specific steganalysis capability of LLMs, we fine-tune LLMs with various instructions, while we equip LLMs with domain-agnostic steganalysis capability by exploring appropriate mix-up of various prevalent stego texts. To achieve acceptable training costs, we adopt the lightweight training strategy of LLMs, which only requires a single GPU. Extensive experimental results show that after lightweight fine-tuning LLMs obtain the state-of-the-art linguistic steganalysis performance, in both domain-specific and domain-agnostic scenarios.

We conduct extensive experiments on exploring the LLMs for linguistic steganalysis, and we find:

- The ability to detect stegos is just embedded in LLMs and fine-tuning can activate this ability. LLMs with instruction fine-tuning achieve much higher accuracy in detecting steganographic text than using the steganalysis model with BERT, while the trainable parameters are much less than with BERT.

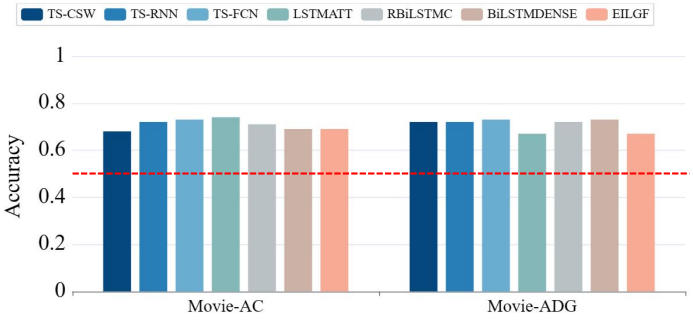
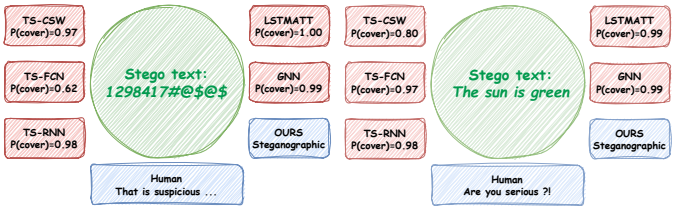



Fig. 1. The detection accuracy of recently representative steganalysis methods[24–26, 33, 34, 39, 40]. Here we test two popular codebook construction methods, namely source coding-based method AC [13] and distribution preserving method ADG [8]. The red dotted line represents the 0.5 accuracy guess line.




(a) An example of low-quality stego (b) An example of irrational stego

Fig. 2. Two "difficult-to-detect" cases. Current steganalysis techniques, such as TS-CSW [25], TS-FCN [24], TS-RNN [26], LSTMATT [34], and GNN [27], produce inaccurate results with high levels of confidence for these two types of steganographic text, while our approach maintains its accuracy. In human opinion, these types of stego are unreadable or irrational, thus leading to suspicion from monitors.

- Based on extensive pre-training datasets, our models exhibit exceptional detection capabilities across diverse steganographic algorithms and datasets. The proficiency stems from the LLMs' capacity to evaluate the fluency and rationality of the text.
- Leveraging the cross-domain capability of LLMs, a general steganalysis model is trained on a blender of datasets. In most scenario, the performance of our general model on unseen dataset is superior to baseline methods trained on this dataset.

## II. Related Work

### A. Linguistic Steganography

Linguistic steganography can be classified into three primary categories: retrieval-based, modification-based, and generation-based linguistic steganography.

For retrieval-based text information hiding methods, such as [41] and [42], the fundamental procedure includes encoding samples from a text corpus and subsequently choosing relevant sentences for transmission, depending on the secret messages to be embedded. Nevertheless, the need for pre-shared codebooks among communicating parties restricts its capacity and reduces its flexibility.

The fundamental concept of modification-based linguistic steganography involves making subtle changes to a given text by using secret messages as control signals. Techniques like

synonym substitution [43] and sentence paraphrasing [44] have been explored.

In recent years, generative linguistic steganography methods can be classified into the following categories:

A. Coding methods based on rules, such as the early approach [12], involve designing predefined rules to directly encode tokens. However, these methods solely rely on rule design and lack theoretical proof for statistical imperceptibility.

B. Coding methods that modify the distribution, such as truncation. This method first disrupts the conditional probability distribution and then source coding techniques like fixed-length coding (FLC) [6], Huffman coding (HC) [15, 16], and arithmetic coding (AC) [13, 14] are applied to the disrupted distribution. These methods greatly undermine the statistical covertness of the steganographic text as they disturb the distribution.

C. Coding methods that preserve the distribution aim to ensure provable imperceptibility under KL divergence by achieving consistency in the distribution between steganographic and natural text. To achieve this goal, researchers have considered distribution shifting [10], entropy coupling [9], grouping [8], and mapping [14].

### *B. Linguistic Steganalysis*

The first-generation steganalysis methods were primarily based on heuristic statistical features. These studies [22, 23] focused on modification-based steganography methods, such as synonym substitution techniques that were popular at that time. However, these methods became less effective with the continuous improvement of language models.

In recent years, second-generation steganalysis methods [24–26, 29–32, 34, 35] have focused on generative linguistic steganography. These methods employ deep learning models to differentiate between steganographic and natural texts using statistical features [24]. The attention mechanism is utilized in [24] to extract the relationship between word embeddings, followed by the use of a classification network with a single activation unit to output the classification results. TS-CSW [25] proposes a model based on convolutional sliding windows, which can extract correlation features using multiple window sizes. Additionally, TS-RNN [26] observes that the conditional probability distribution of each word in steganographic texts becomes distorted after secret messages are embedded. To address this, the method proposed in [26] employs recurrent neural networks to extract these distribution differences and subsequently classify texts as non-steganographic or steganographic.

Due to the limited classification ability of simple neural networks, subsequent studies adopted increasingly complex deep learning models. A model based on a graph neural network (GNN) was proposed by [27], where texts were transformed into graphs. In these graphs, nodes represented words, and edges captured associations between them. EILGF [33] focused on extracting and integrating local semantic features and global long-term dependencies to enhance the quality of text representation. The improved representation facilitated more accurate classification. [32] involved fine-tuning a BERT extractor through the hierarchical supervised learning that combines signals from multiple softmax classifiers, rather than relying solely on the final one. [31] combined the cross-entropy loss and the supervised contrastive loss to guide model training to obtain better results.

With the development of text steganography techniques, differentiating statistical features between steganographic and natural texts has become more challenging. This difficulty arises due to the existing deep learning models' struggle to capture the intricate and overlapping boundaries between the two types of text. Moreover, as LLMs can now generate text that is highly similar to natural text, the effectiveness of current deep learning methods is gradually declining.

The potential solution lies in constructing the third-generation steganalysis methods based on LLMs, leveraging their powerful capabilities to develop a specialized model in steganalysis.

### *C. Large Language Models*

Commonly, large language models (LLMs) are referred to as language models with billions of parameters that are trained on massive amounts of text. Examples of such models include GPT-3 [45], PaLM [46], BLOOM [47], LLaMA [48] and so on. LLMs are specifically constructed using the Transformers [49] architecture, which stacks multiple self-attention layers in a deep network.

LLMs possess excellent natural language understanding abilities and can produce high-quality text based on given prompts. They can perform tasks such as generating code, converting formatted data into JSON files, and extracting key information from articles. Due to their powerful functionalities, smaller language models like Alpaca [50] and Vicuna [51] even utilize API interfaces to directly access data from ChatGPT and GPT-4 [52] for training, aiming to attain similar language capabilities. According to a previous study [38], using ChatGPT for steganalysis tasks achieved comparable results to the second-generation steganalysis method without BERT, with just 32 examples in the prompt. Training LLMs on a dataset containing steganographic and natural texts is expected to yield improved steganalysis results.

Directly training LLMs can be costly and complex, while fine-tuning LLMs is the most frugal option. The current fine-tuning method, LoRA [53], decomposes the parameter matrix of the model into two low-rank matrices, and creates a parallel LoRA branch of these matrices. The outputs of the LoRA branch are then merged with the hidden layers output of the original model. During training, the parameters of the original model are frozen, and only the two low-rank matrices in the LoRA branch are trained. This significantly reduces the parameter size, often to 1% or lower. In most cases, the effects of LoRA fine-tuning and full fine-tuning are comparable. Considering resource efficiency and convenience, this paper adopts the LoRA method to fine-tune the LLMs.

## III. Methodology

### *A. Models & Config*

We employ LoRA [53] to fine-tune the LLMs and directly get the detection results from the LLMs. We utilized the

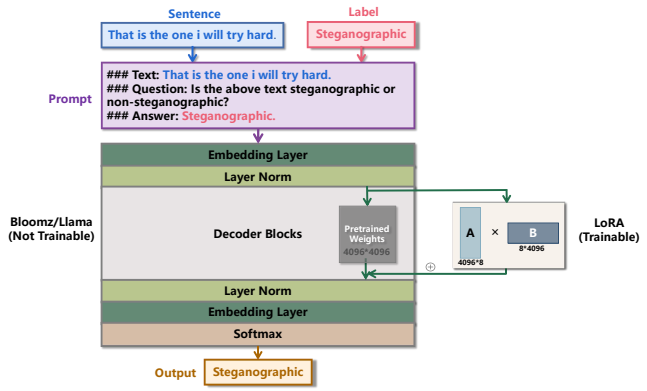



Fig. 3. The architecture for instruction fine-tuning in Bloomz/Llama + LoRA. Input sentences and labels are filled into a reasonable prompt template for LLM's fine-tuning. The training of parameters is limited to only the LoRA branch, while the parameters of LLMs remain constant.

TABLE I
TRAINABLE PARAMETERS

| Model | Trainable parameters | Total parameters | Ratio |
|---|---|---|---|
| Llama-7B+LoRA | $4,194,304$ | $6,742,609,920$ | $0.0622\%$ |
| Bloom-7B1+LoRA | $3,932,160$ | $7,072,948,224$ | $0.0556\%$ |
| BERT-base | $\sim 110,000,000$ | $\sim 110,000,000$ | $100\%$ |

Bloomz-7B1 [54] and the Llama-7B [48]. Both models are openly available in https://huggingface.co/. To account for our limited GPU capabilities and the trade-off between LoRA hyperparameters and training cost, we opted for LLMs with training parameters smaller than 7B. Larger models will make our GPUs out of memory. To guarantee reliable and pragmatic experimental findings, we opted for the most sizeable LLMs that our GPUs could support - the Bloomz-7B1 and Llama-7B (or Bloomz/Llama for convenience).

The structure of LLMs with LoRA fine-tuning is illustrated in Fig. 3. We froze all LLM parameters during the training process and exclusively trained the parameters of the LoRA branch. Thus, the NLL (Negative Log Likelihood) loss of LLM can be utilized to optimize the detection ability. Moreover, since the LLM parameters were not trained, the ability of original LLM would never decrease. This fine-tuning method guaranteed that our models could leverage the full human-like capability of original LLMs to make more precise detection.

With the LoRA fine-tune framework, trainable parameters of LLMs can be reduced to less than $0.1\%$, which is much less than that of BERT. Table I shows the trainable parameters of our models and BERT-base.

Fixed hyperparameters used in training is presented in Table II. According to the report from BigScience [54], the learning rate of Bloomz is set to 2e-5 during the multi-task fine-tuning phase. Hence, the learning rate for any further fine-tuning should not surpass 2e-5. The *batch size* is set to 4 and the *lora rank* is set to 8, which signifies the optimal configuration for executing the Bloomz-7B1 model on a single RTX3090 GPU. Under these conditions, the model has displayed robust performance, exhibiting the potential of LLMs in steganalysis tasks. The hyperparameter selection process of Llama is similar to

TABLE II
FIXED TRAINING HYPERPARAMETERS

| Model | Bloomz | Llama |
|---|---|---|
| batch size | 4 | 4 |
| max learning rate | 1e-5 | 1e-5 |
| lora rank | 8 | 8 |
| lora alpha | 16 | 16 |
| lora dropout | 0.05 | 0.05 |

TABLE III
AVERAGE PERPLEXITY AND TOKENS OF THE DATASETS

| Features | Tweet-Natural | Tweet-AC | Tweet-HC | Tweet-ADG |
|---|---|---|---|---|
| Tokens | 10.66 | 10.78 | 9.44 | 9.74 |
| PPL | 946.16 | 1016.45 | 635.35 | 1099.58 |
| Features | Movie-Natural | Movie-AC | Movie-HC | Movie-ADG |
| Tokens | 25.63 | 26.48 | 24.00 | 27.80 |
| PPL | 254.33 | 261.51 | 119.20 | 256.12 |
| Features | News-Natural | News-AC | News-HC | News-ADG |
| Tokens | 23.10 | 26.10 | 24.42 | 25.93 |
| PPL | 253.65 | 242.22 | 109.87 | 248.75 |

that of Bloomz. Further refinement of these hyperparameters may improve results. The previously mentioned parameter settings are intended to minimize training time and demand for GPU capacity.

### B. Datasets

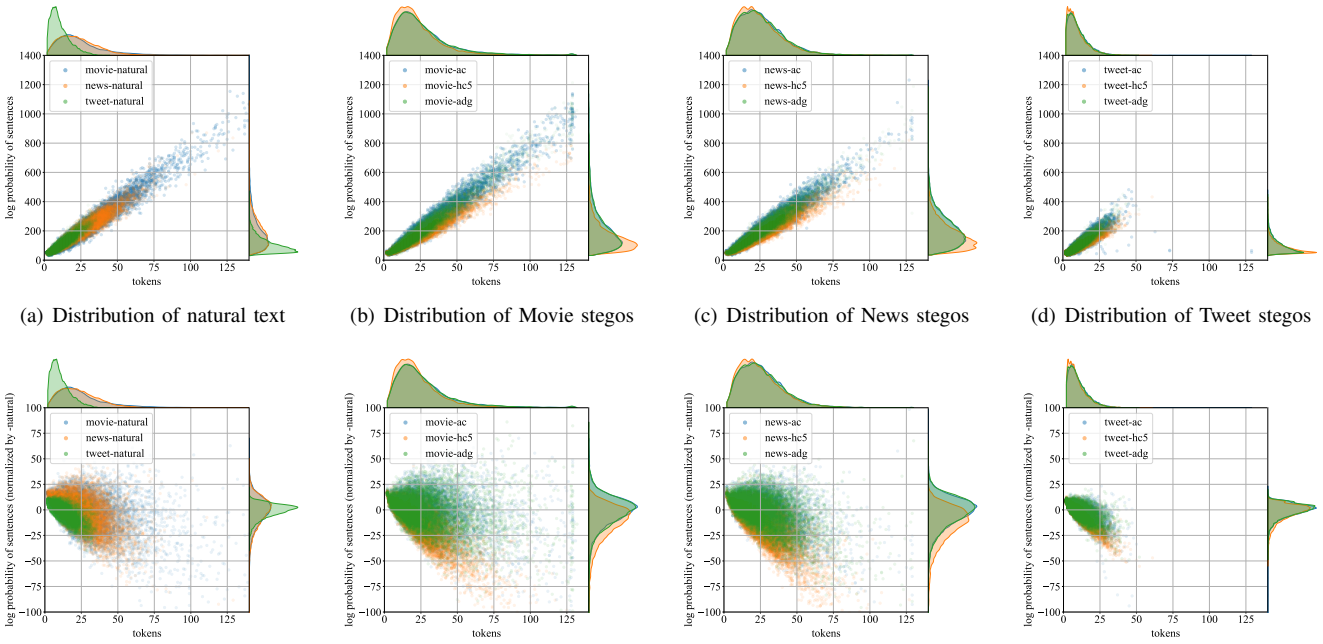
We utilize three datasets: Tweet, Movie, and News. We have trained copies of GPT-2 models on each dataset. Steganographic texts are generated using three different embedding methods: AC(Arithmetic Coding) [13], HC(Huffman Coding) [16], and ADG(Adaptive Dynamic Grouping) [8]. We randomly sampled 10,000 sentences each from natural text and steganographic-generated text to construct our datasets. These datasets are labeled as Movie-Natural, Movie-AC, Movie-HC, Movie-ADG, Tweet-Natural, Tweet-AC, Tweet-HC, Tweet-ADG, News-Natural, News-AC, News-HC, and News-ADG. The first part of each label represents the data source, and the second part represents the steganographic encoding algorithm. Table III presents the statistical features of each dataset.

We modeled the distributions of datasets in terms of sentence length and perplexity. Sentence length is the number of tokens that the text breaks down under the tokenizer of the model. And perplexity (PPL) is a widely employed metric to assess the fluency of text, whereas entropy represents a statistical measure of the information content. The formulas for PPL and entropy can be expressed as follows.

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(x_i|\mathbf{x}_{1:i-1})\right) \tag{1}$$

where $N$ represents the total number of tokens, and $p(x_i)$ denotes the conditional probability assigned to each token $x_i$.

It is evident that the statistical characteristics of the Tweet dataset differ significantly from those of the Movie and News datasets. The average length of the Tweet is less than half of the average length observed in the Movie and News datasets, whereas the PPL is nearly 3 times higher. This suggests that the texts in Tweet are disorganized and confused. In




(a) Distribution of natural text (b) Distribution of Movie stegos (c) Distribution of News stegos (d) Distribution of Tweet stegos

(e) Normalized distributions of natural text (f) Normalized distributions of Movie stegos (g) Normalized distributions of News stegos (h) Normalized distributions of Tweet stegos

Fig. 4. Distributions of text, estimated by log probability of sentences and the number of tokens in sentences. Each figure consists of the main scatter plot of joint distribution and lines of marginal distributions at top and right. (a) represents the distributions of 3 natural datasets, and (b) (c) (d) denote the distributions of AC/HC/ADG stegos within these datasets, respectively. The second row illustrates the normalized distribution, estimated by normalized log probability of sentences and the number of tokens in sentences, corresponding to the first row.

contrast, the statistical characteristics of Movie and News exhibit relatively similar patterns.

The three steganographic encoding algorithms display distinct characteristics. The steganographic texts created by HC possess the lowest PPL, surpassing even natural texts. Conversely, the steganographic texts produced by AC closely emulate the statistical properties of natural texts. In the case of ADG-generated texts, they demonstrate a similar pattern to AC-generated texts but with a higher perplexity.

It should be noted that PPL is not considered the definitive criterion for assessing the quality of text generation. Upon manual evaluation, both the steganographic texts produced by AC and ADG exhibit considerable similarity. PPL merely offers a concise overview of the distribution of text in the dataset and is inadequate for discerning between high and low-quality text.

To more accurately characterize the distribution of the dataset, we calculated the log probability of sentences, then used log probability as the x-axis and sentence length (counted by tokens) as the y-axis to model the distribution of text. The distribution of log probability and sentence length can be viewed as a simplified 2-dimensional text probability distribution. The log probability can be calculated as Equation 2:

$$log\,probability = -\sum_{i=1}^{N} \log p(x_i|\mathbf{x}_{1:i-1}), \tag{2}$$

where $N$ denotes the length of the sentence.

As illustrated in Fig. 4(a), 4(b), 4(c), and 4(d), the distribution of the Tweet dataset is primarily focused on short sentences with a relatively high log probability. Although the distributions of the Movie and News datasets are similar, the main discrepancy lies in the fact that the long-tailed distribution of the Movie dataset constitutes a larger proportion.

To elucidate the influence of steganographic algorithms on the distribution of text, we standardized the logarithmic probability of AC/HC/ADG using the mean and standard deviation of natural text. Based on Fig. 4(e),4(f),4(g),4(h), it is clear that the distribution of stegos produced by HC is lower than that of the other distributions. The distribution of edges in HC stegos reveals that HC tends to produce text that significantly deviates from natural human writing patterns. This is due to its peak position of log probability is less than 0, resulting in a noticeable distinguishability of HC-generated stegos. Conversely, AC and ADG stegos display a nearly identical distribution; however, the AC distribution appears to be more focused than that of ADG.

### C. Evaluation Metrics

We denote the output results of the model as TS (True Steganographic), FS (False Steganographic), US (Unknown Steganographic), TN (True Non-Steganographic), FN (False Non-Steganographic), and UN (Unknown Non-Steganographic), where US and UN represent samples that do not match the output template. The occurrence of US and UN is not unusual in [38], but in our models they only appear when the models are not fine-tuned. We primarily use accuracy and F1 score as evaluation metrics. Since the existence of US and UN samples, redefining the calculation of metrics is necessary. The accuracy is redefined as follows:

TABLE IV
RESULTS ON MOVIE, NEWS AND TWEET. "W/O FT" REPRESENTS THE ZERO-SHOT OUTPUT WITHOUT FINE-TUNING. THE BASELINE MODELS [24–26, 33, 34, 39, 40] AND OUR MODELS ARE TRAINED FOR 10 EPOCHS ON EACH DATASET.

| Models | Movie-AC | | | | Movie-HC | | | | Movie-ADG | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall |
| TS-CSW [25] | 68.17 | 67.92 | 71.27 | 64.87 | 79.05 | 80.23 | 75.30 | 85.86 | 71.80 | 68.81 | 76.09 | 62.80 |
| TS-RNN [26] | 72.20 | 69.31 | 76.44 | 63.40 | 81.15 | 81.75 | 78.52 | 85.26 | 72.03 | 68.93 | 76.60 | 62.65 |
| TS-FCN [24] | 73.35 | 72.01 | 75.04 | 69.21 | 81.15 | 81.89 | 78.10 | 86.07 | 72.80 | 70.45 | 76.25 | 65.47 |
| LSTMATT [34] | 74.40 | 72.57 | 77.30 | 68.40 | 82.00 | 82.95 | 78.11 | 88.44 | 66.88 | 65.32 | 67.82 | 63.00 |
| RbiLSTMC [39] | 70.88 | 67.72 | 75.06 | 61.68 | 78.90 | 80.10 | 75.14 | 85.76 | 72.03 | 68.96 | 76.54 | 62.75 |
| BiLSTMDENSE [40] | 69.30 | 59.55 | 87.34 | 45.18 | 80.65 | 81.70 | 76.83 | 87.23 | 73.05 | 70.19 | 77.61 | 64.06 |
| EILGF [33] | 69.42 | 67.83 | 72.88 | 63.44 | 76.67 | 75.57 | 74.40 | 76.77 | 66.50 | 64.61 | 68.22 | 61.37 |
| ChatGPT (32 shots) [38] | 66.60 | 73.58 | 60.86 | 93.00 | 65.80 | 68.04 | 64.02 | 72.60 | 70.00 | 72.58 | 66.84 | 79.40 |
| Bloomz w/o FT | 43.65 | 51.85 | 45.48 | 60.30 | 43.70 | 51.91 | 45.52 | 60.40 | 43.30 | 51.40 | 41.85 | 59.60 |
| Llama w/o FT | 52.50 | 60.87 | 51.79 | 73.80 | 53.70 | 62.23 | 52.59 | 76.20 | 53.25 | 61.72 | 52.29 | 75.30 |
| Bloomz w/ FT | 86.50 | 86.53 | 86.35 | 86.70 | 88.85 | 89.14 | 86.86 | 91.55 | 86.05 | 86.05 | 86.05 | 86.05 |
| Llama w/ FT | **89.35** | **89.55** | 87.87 | 91.30 | **90.95** | **91.01** | 90.42 | 91.60 | **89.15** | **89.08** | 89.67 | 88.50 |
| Models | News-AC | | | | News-HC | | | | News-ADG | | | |
| | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall |
| TS-CSW [25] | 70.25 | 68.28 | 72.33 | 64.66 | 77.90 | 79.64 | 73.23 | 87.28 | 70.00 | 67.62 | 72.64 | 63.25 |
| TS-RNN [26] | 70.88 | 67.70 | 61.64 | 61.64 | 79.08 | 80.16 | 75.56 | 85.36 | 70.15 | 66.52 | 74.82 | 59.87 |
| TS-FCN [24] | 71.02 | 69.03 | 73.32 | 65.22 | 79.18 | 80.93 | 74.06 | 89.20 | 71.50 | 67.89 | 76.80 | 60.83 |
| LSTMATT [34] | 67.77 | 65.96 | 69.16 | 63.05 | 74.55 | 74.95 | 73.11 | 76.88 | 71.95 | 69.14 | 75.95 | 63.45 |
| RbiLSTMC [39] | 70.30 | 66.80 | 74.83 | 60.32 | 79.30 | 80.26 | 76.05 | 84.96 | 69.60 | 66.01 | 73.95 | 59.61 |
| BiLSTMDENSE [40] | 68.75 | 63.41 | 75.47 | 54.67 | 78.10 | 79.97 | 73.08 | 88.28 | 69.88 | 66.92 | 73.34 | 61.53 |
| EILGF [33] | 61.83 | 63.53 | 62.74 | 64.35 | 73.58 | 73.39 | 75.61 | 71.29 | 61.17 | 61.42 | 62.88 | 60.03 |
| ChatGPT (32 shots) [38] | 66.30 | 71.37 | 62.04 | 84.00 | 59.30 | 56.28 | 60.79 | 52.40 | 67.30 | 66.39 | 68.51 | 64.40 |
| Bloomz w/o FT | 47.40 | 45.05 | 47.56 | 42.80 | 49.45 | 48.33 | 49.84 | 46.90 | 48.55 | 46.91 | 48.86 | 45.10 |
| Llama w/o FT | 49.10 | 58.71 | 49.42 | 72.30 | 51.15 | 61.02 | 50.80 | 76.40 | 49.35 | 59.00 | 49.59 | 72.80 |
| Bloomz w/ FT | 82.85 | 82.80 | 83.05 | 82.55 | 88.33 | 88.45 | 87.48 | 89.45 | 83.53 | 83.92 | 81.94 | 86.00 |
| Llama w/ FT | **87.55** | **87.74** | 86.42 | 89.10 | **89.95** | **89.94** | 89.99 | 89.90 | **88.45** | **88.59** | 87.51 | 89.70 |
| Models | Tweet-AC | | | | Tweet-HC | | | | Tweet-ADG | | | |
| | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall |
| TS-CSW [25] | 66.02 | 64.64 | 66.70 | 62.69 | 71.30 | 73.28 | 67.99 | 79.45 | 64.58 | 63.93 | 64.48 | 63.40 |
| TS-RNN [26] | 66.72 | 33.19 | 33.32 | 65.77 | 72.15 | 73.50 | 69.50 | 77.99 | 65.22 | 64.68 | 65.07 | 64.31 |
| TS-FCN [24] | 68.25 | 67.43 | 68.52 | 66.38 | 71.23 | 73.71 | 67.31 | 81.47 | 66.63 | 66.52 | 66.10 | 66.93 |
| LSTMATT [34] | 63.20 | 60.79 | 64.35 | 57.60 | 68.38 | 70.21 | 65.80 | 75.26 | 64.67 | 63.41 | 76.44 | 70.62 |
| RbiLSTMC [39] | 67.35 | 67.05 | 67.02 | 67.09 | 71.73 | 73.96 | 67.99 | 81.07 | 66.10 | 65.16 | 65.35 | 64.01 |
| BiLSTMDENSE [40] | 66.70 | 67.11 | 65.68 | 68.60 | 70.63 | 73.13 | 66.85 | 80.72 | 65.63 | 65.15 | 65.43 | 64.87 |
| EILGF [33] | 68.00 | 67.07 | 69.20 | 65.06 | 69.75 | 64.52 | 74.66 | 56.80 | 67.17 | 67.60 | 66.40 | 68.84 |
| ChatGPT (32 shots) [38] | 61.10 | 63.75 | 59.69 | 68.40 | 48.70 | 57.24 | 49.42 | 68.00 | 56.20 | 65.07 | 54.11 | 81.60 |
| Bloomz w/o FT | 54.95 | 63.61 | 53.56 | 78.30 | 56.30 | 65.09 | 54.40 | 81.00 | 54.10 | 62.66 | 53.01 | 76.60 |
| Llama w/o FT | 49.90 | 65.66 | 49.95 | 95.80 | 50.85 | 66.53 | 50.44 | 97.70 | 49.60 | 65.38 | 49.79 | 95.20 |
| Bloomz w/ FT | 76.78 | 76.84 | 76.63 | 77.05 | 79.73 | 80.57 | 77.33 | 84.10 | 75.08 | 75.13 | 74.96 | 75.30 |
| Llama w/ FT | **79.30** | **79.84** | 77.80 | 82.00 | **82.35** | **82.62** | 81.38 | 83.90 | **78.40** | **79.13** | 76.54 | 81.90 |

$$Acc = \frac{TS + TN}{TS + FS + US + TN + FN + UN}. \tag{3}$$

Due to the equitable sampling of steganographic and natural texts in our dataset, and taking into account the balance between recall and precision, we utilize the F1 score with $\beta = 1$ as the principal evaluation metric for steganalysis. The F1 score is obtained as follows:

$$Precision = \frac{TS}{TS + FS + US}, \tag{4}$$

$$Recall = \frac{TS}{TS + FN}, \tag{5}$$

$$F1 = (1 + \beta^2) \cdot \frac{Precision \cdot Recall}{\beta^2 \cdot Precision + Recall}, \beta = 1. \tag{6}$$

The expression for F1 can be obtained by substituting the formulas for Precision and Recall, as follows:

$$F1 = \frac{2 \cdot TS}{2 \cdot TS + FN + FS + US}. \tag{7}$$

### D. Prompt Design

We primarily employ the following templates to construct the instruction fine-tuning dataset:

*### Text:*{*input sentence*}

*### Question: Is the above text steganographic or non-steganographic?*

*### Answer:* {*input label*}

The effects of the prompts and their impact on steganalysis will be thoroughly discussed in Section IV-D.

## IV. EXPERIMENTS AND ANALYSIS

To verify the effectiveness and generality of our method, we first tested in domain-specific steganalysis task and compared the results with current advanced steganalysis methods. Then we analyzed the error samples of this task and tried to find some clues for training a general steganalysis model. After that, we tested our methods in a more practical and challenging scenario of domain-agnostic steganalysis. To figure out the critical factors that affected the performance of our method, we investigated the detection accuracy on sentences of various

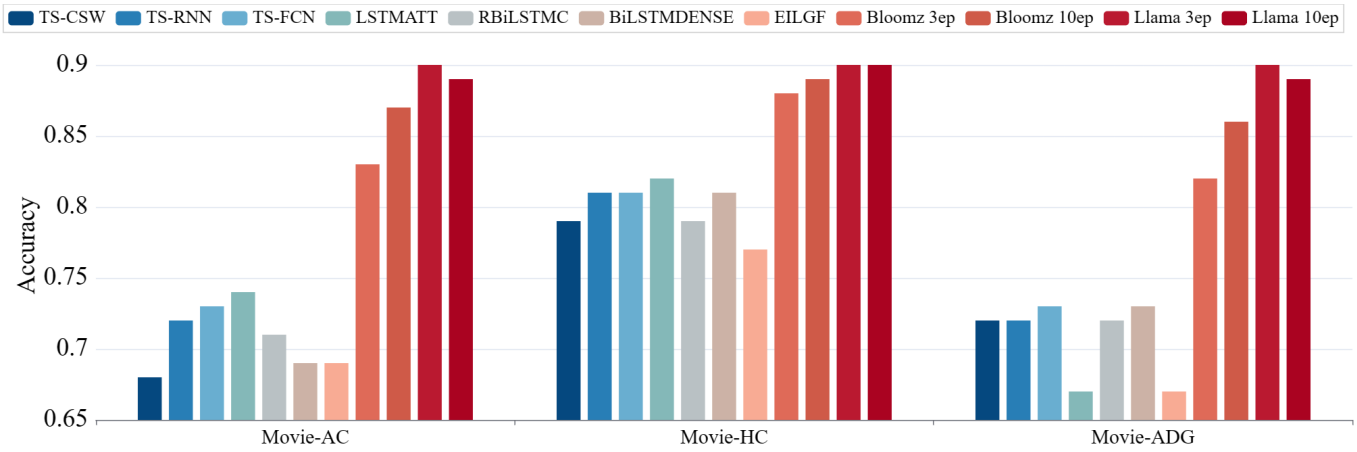



Fig. 5. Detection accuracy of Bloomz/Llama training with 3/10 epochs and baseline methods training with 10 epochs on Movie datasets.

length and different training prompts. Finally, we trained a general steganalysis model that was effective in diverse tasks.

The baseline steganalysis methods used for comparison are as follows: TS-FCN [24] utilizes a single-layer fully connected network to discern statistical correlation at the word level. TS-RNN [26] employs a two-layer bidirectional RNN to decide on temporal connections between words. TS-CSW [25] incorporates a multi-scale kernel CNN in its discriminator, enabling the recognition of statistical features of various sizes. RBiLSTMC [39] combines the advantages of Bi-LSTM and CNN to enhance detection accuracy. LSTMATT [34] utilizes the attention mechanism. BiLSTMDENSE [40] improves low-level features through dense connections and feature pyramids. EILGF [33] extracts and integrates local and global features.

These methods encompass classical and efficient steganalysis models, as well as the latest and state-of-the-art steganalysis models.

### A. Domain-Specific Steganalysis

We conduct domain-specific steganalysis experiments within a dataset of steganographic texts and a corresponding dataset of natural texts. These 2 datasets are divided into training, validation, and test sets in a ratio of 3:1:1. This ratio is consistently maintained for all subsequent experiments.

The Bloomz/Llama and the baseline models were compared on all datasets, as shown in Table IV. The result show that without fine-tuning the LLMs are failed to demonstrate the capability of steganalysis. After fine-tuning the detection accuracy and F1 score of LLMs outperform all baseline methods on all datasets.

Fig. 5 shows a comparison of the Bloomz training with 3/10 epochs, Llama training with 3/10 epochs, and the baseline methods training with 10 epochs. Results show that both Bloomz and Llama consistently outperform the baseline methods in all scenarios, with significant improvements in AC and ADG. It is imperative to clarify that the validation set utilized in our experiments was only used for comparison with baseline methods and not incorporated by our models.

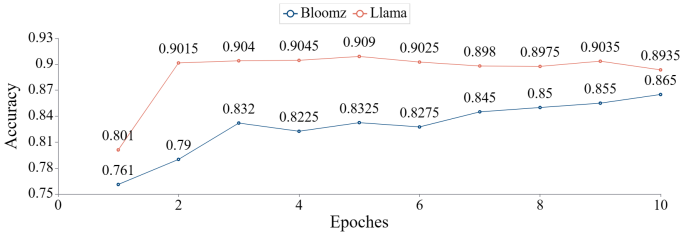


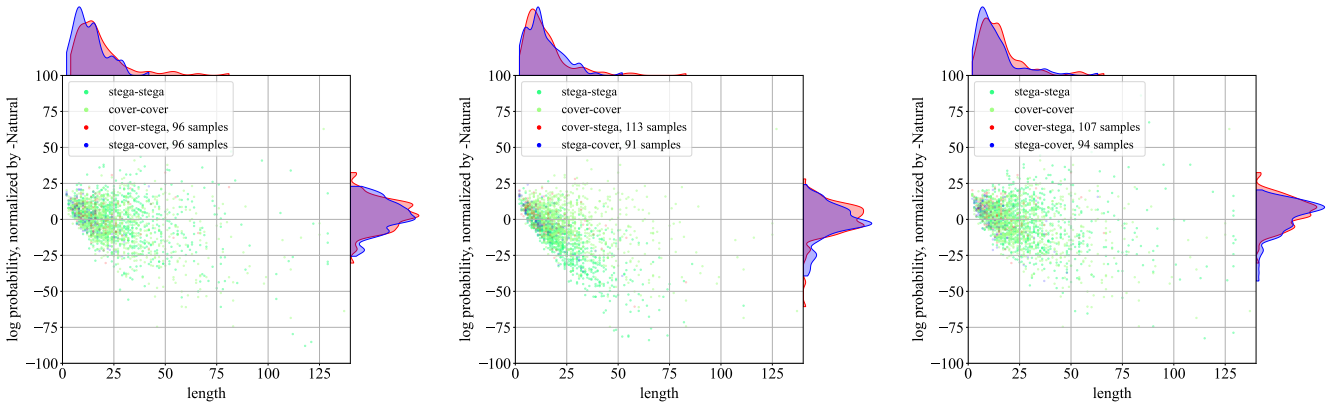

Fig. 6. Detection accuracy of Bloomz/Llama training with 1-10 epochs on Movie-AC dataset.

The results obtained from the Movie-ADG dataset reveal that Bloomz yields a superior accuracy rate of approximately 13% and an F1 score that is around 16% higher than the optimal baseline model. Moreover, the more effective Llama poses an accuracy rate that is roughly 16% higher and an F1 score of approximately 19%, with a detection success rate of nearly 90%. Furthermore, detection accuracy on the three steganographic encoding algorithms achieved at least 90% in the Movie dataset, while not appearing the discrepancy seen in the baseline methods, where detection accuracy on HC-generated texts was notably higher than that of AC and ADG.

In Llama's experiments, it was noted that in particular datasets (Movie-AC, Movie-ADG, News-HC, News-ADG) training for 3 epochs led to superior outcomes than training for 10 epochs. This phenomenon could be attributed to the repetition of training data. Our models' results of training for 1-10 epochs on the Movie-AC dataset are illustrated in Fig. 6. The figure indicates that Bloomz and Llama outperformed the BERT baseline models with just one epoch of training. Llama attained its highest accuracy in the fifth epoch, whereas it displayed a decreasing trend in accuracy with further training. On the other hand, Bloomz showed a gradual enhancement in accuracy over 10 epochs. Overall, Llama exhibited a detection accuracy that was about 3-5% higher than Bloomz.

### B. Error Analysis of Domain-Specific Steganalysis

To analyze the sentences that are non-detected (stegos that are not detected by our models) and incorrectly detected



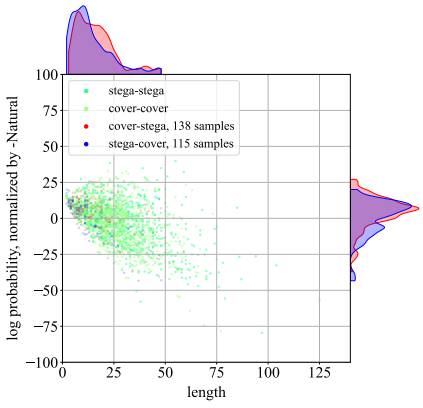

(a) Distribution of error samples in Movie-AC



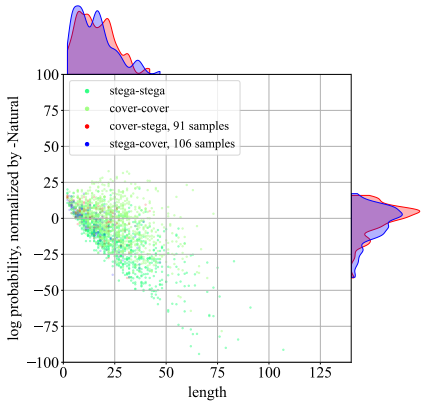

(b) Distribution of error samples in Movie-HC



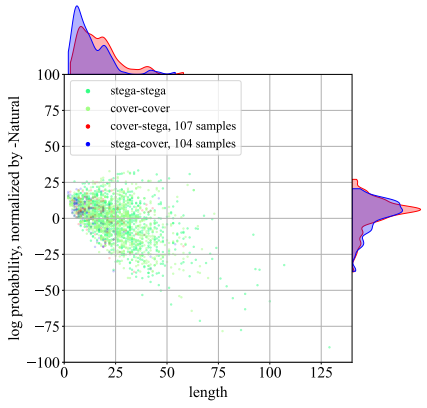

(c) Distribution of error samples in Movie-ADG



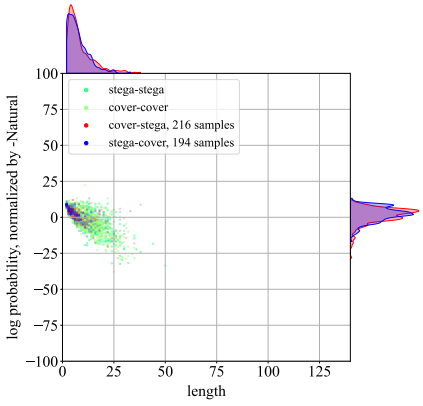

(d) Distribution of error samples in News-AC



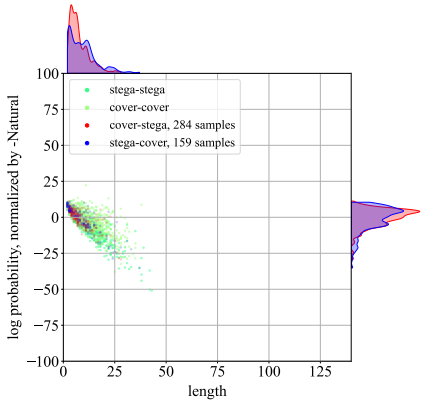

(e) Distribution of error samples in News-HC



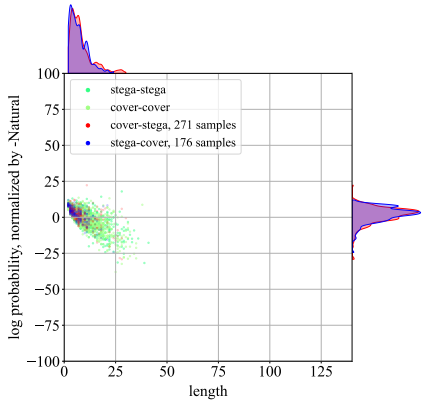

(f) Distribution of error samples in News-ADG




(g) Distribution of error samples in Tweet-AC




(h) Distribution of error samples in Tweet-HC



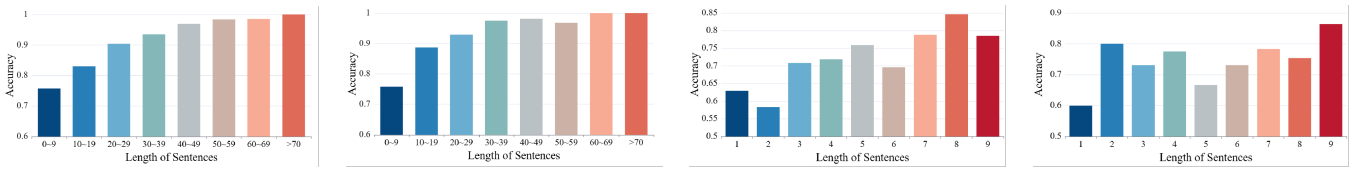

(i) Distribution of error samples in Tweet-ADG

Fig. 7. Distributions of error samples in domain-specific steganalysis. Each figure consists of the main scatter plot of joint distribution and lines of marginal distributions of error samples at top and right.

(covers that are wrongly detected as stegos by our models) in the domain-specific tasks, we scatter their distributions in Fig. 7.

In most cases, the normalized probability of the error samples is similar to the -Natural dataset, since their marginal distribution of normalized probability is concentrated near 0. The lengths of these samples are shorter than the average of -Natural dataset. Precisely, we find that for sentences of the same length, the lower the PPL the more likely it is to be misclassified by the model. From the distributions, it can be observed that a large proportion of texts is concentrated in a small region, and the error samples are located at the edge of this region. This phenomenon reminds us that these short and extremely fluent stegos are challenging samples for our models, which also proves that our models take fluency and rationality of text as standards of judgement. Most of the results show that the incorrectly detected samples are more than non-detected samples, which might be a warning sign of over-fitting.

The detection accuracy of our models varies based on



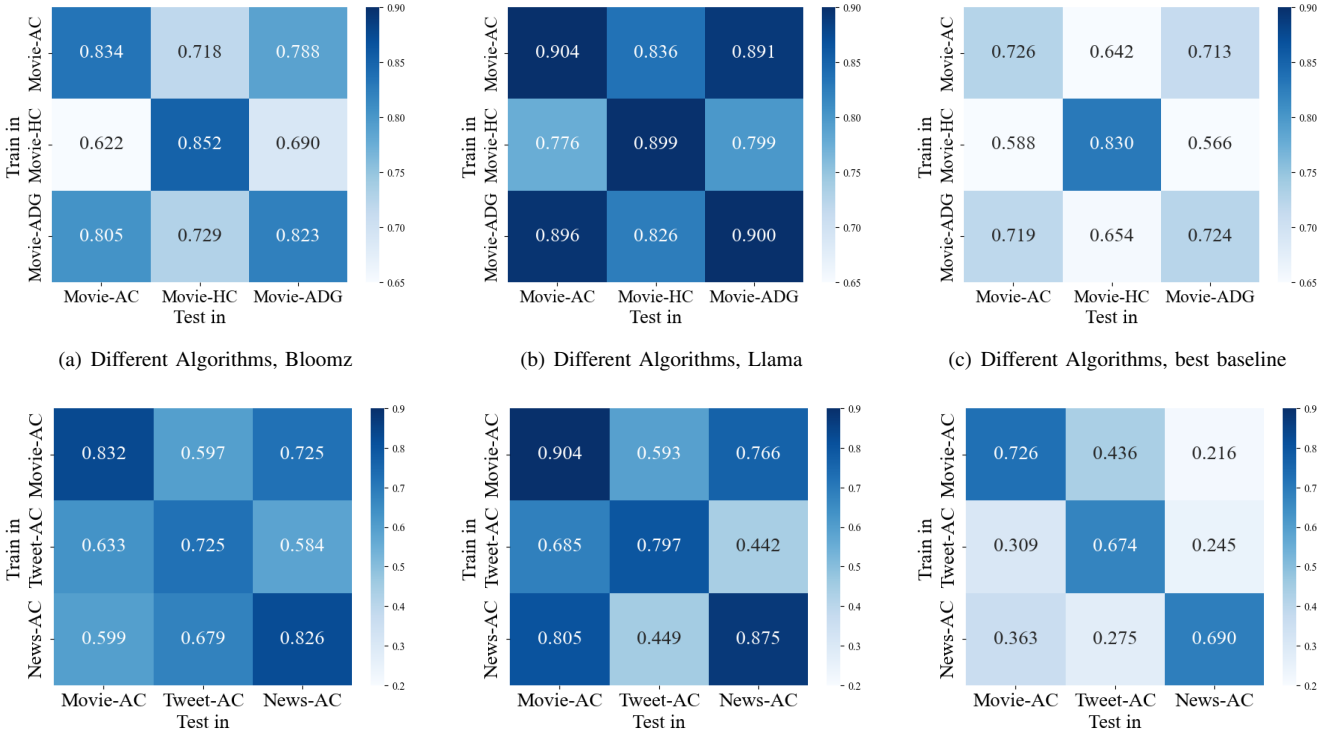

(a) Detection accuracy of Bloomz on different length texts.

(b) Detection accuracy of Llama on different length texts.

(c) Detection accuracy of Bloomz on texts of less than 10 tokens.

(d) Detection accuracy of Llama on texts of less than 10 tokens.

Fig. 8. Detection accuracy of Bloomz/Llama on sentences of various lengths.




(a) Different Algorithms, Bloomz

(b) Different Algorithms, Llama

(c) Different Algorithms, best baseline

(d) Different Sources, Bloomz

(e) Different Sources, Llama

(f) Different Sources, best baseline

Fig. 9. F1 scores of Bloomz, Llama and baseline mpdels on the domain-agnostic steganalysis task. y-axis denotes the training dataset and x-axis denotes the test dataset.

TABLE V
SHORT TWEET SAMPLES. THE RIGHTMOST COLUMN REPRESENTS THE DETECTION RESULTS OF A HYBRID EXPERT MODEL CONSISTING OF BASELINE METHODS [24–27, 34].

| Sentence | True label | Pred. of ours | Pred. of [24–27, 34] |
|---|---|---|---|
| Yep thats a good thing | stego | stego | cover |
| Bit worried my poor pup | stego | cover | cover |
| Lol | cover | cover | cover |
| No I did not get it | cover | cover | stego |
| Yay | cover | stego | cover |

lengths of sentences. Fig. 8(a) and 8(b) show that sentences with less than 10 tokens have a detection rate of less than 80%. However, sentences with over 20 tokens have an accuracy of 90% or more. Additionally, sentences with more than 70 tokens have a detection rate of 100%, suggesting that our models are nearly faultless in those cases.

There are some short samples presented in Table V. It is almost impossible for humans to differentiate between these fragmented but authentic stegos and covers, and the mix-up of baseline methods is also struggling. The broad range of tweets does help to generate seemingly innocent steganographic texts, and the restricted length of tweets presents a greater obstacle for detection models.

### C. Domain-Agnostic Steganalysis

We evaluated the domain-agnostic capability of two LLMs across various steganographic algorithms and datasets. This task consisted of training on one coding algorithm and testing on two other algorithms, or training on one type of dataset and testing on other datasets. Each experiment entailed 3 epochs of training on the dataset. The outcomes are displayed in Fig. 9(a), 9(d), 9(b), 9(e). All of the baseline methods mentioned above were also tested, and Fig. 9(c), 9(f) show the highest F1 scores achieved by these models.

Our experiment has demonstrated the superiority of our models in terms of its domain-agnostic ability compared with baseline methods, surpassing the performance of baseline

TABLE VI
RESULT OF DIFFERENT PROMPTS, TRAINING FOR 3EPOCHES IN TWEET-AC. FOR THE 7TH PROMPT, OUR MODELS GENERATES NON-STANDARD OUTPUTS AND DOES NOT PERFORM ANY ABILITY OF STEGANALYSIS.

| Prompts | Bloomz-7B1 | | | | Llama-7B | | | |
|---|---|---|---|---|---|---|---|---|
| | Accuracy | F1 score | Precision | Recall | Accuracy | F1 score | Precision | Recall |
| Prompt 1 | 0.725 | 0.701 | **0.768** | 0.645 | 0.795 | 0.797 | 0.789 | 0.806 |
| Prompt 2 | 0.741 | 0.748 | 0.728 | 0.769 | 0.803 | **0.808** | 0.787 | **0.831** |
| Prompt 3 | **0.751** | **0.761** | 0.732 | **0.792** | 0.803 | **0.808** | 0.787 | **0.831** |
| Prompt 4 | 0.688 | 0.658 | 0.727 | 0.600 | **0.805** | 0.807 | **0.798** | 0.816 |
| Prompt 5 | 0.500 | 0.000 | 0.500 | 0.000 | 0.799 | 0.804 | 0.784 | 0.826 |
| Prompt 6 | 0.704 | 0.697 | 0.714 | 0.680 | 0.800 | 0.801 | 0.796 | 0.805 |
| Prompt 7 | - | - | - | - | - | - | - | - |
| Prompt 8 | 0.733 | 0.744 | 0.715 | 0.775 | 0.789 | 0.788 | 0.791 | 0.786 |

models considerably. As a result, we can assume that our models can robustly derive more accurate steganographic characteristics from various sources of text generated by different encoding algorithms. These extracted features can be applied to a wide range of steganographic texts. The distribution of text across various datasets presents a significant challenge to our models' domain-agnostic capability. This is due to certain data being classified as steganographic in the Movie dataset but not necessarily so in the Tweet dataset. Such variations must be taken into account to ensure optimal performance.

Furthermore, the pattern of Bloomz in the transfer experiments on different sources deviates significantly from that of the standard models. The Movie dataset has an average of 25.63 tokens per sentence and a perplexity value of 1059.65, whereas the News dataset has an average of 23.1 tokens per sentence and a perplexity value of 958. Despite having similar sentence lengths and perplexity values, the content of these two datasets is quite different. Bloomz performs significantly better when trained on the Movie-AC dataset and tested on the News-AC dataset, in contrast to the baseline models. Comparable results can be observed from the Movie and Tweet datasets, indicating that our model has learned unique steganographic features in comparison to the baseline models. According to Bloomz, the News dataset may be more comparable to the Movie dataset, whereas the Tweet dataset significantly differs from the Movie dataset. Conversely, the Llama views the News and Movie datasets as having minimal differences. Llama exhibits robust transfer performance between the encoding algorithms, with negligible performance loss between the AC and ADG algorithms. Nonetheless, it encounters considerable performance degradation in transfer tasks across datasets, potentially attributable to the restricted diversity of training data implemented for the Llama model.

However, the Tweet dataset exhibits notably weaker transfer performance than the majority of datasets. As we mentioned before, part of our models' ability is from the perception of text fluency and rationality. While the sentences in Tweet dataset are often fragmented spoken expressions. We assume that the reason can be attributed to the complexity and non-normative nature of Twitter texts.

### *D. Effectiveness of Prompts*

To assess the impact of prompts on models' detection performance, including inputting instruction options (separators, hints and questions) and outputting format options (such as long terms like "steganographic/non-steganographic", short terms like "stega/cover", simple responses like "yes/no", or even numerical answers like "0/1"), we employed the prompts presented in Table VII.

TABLE VII
PROMPTS

| | |
|---|---|
| Prompt 1 | ### Text: {input sentence}<br>### Question: Is the above text steganographic or non-steganographic?<br>### Answer: {input label (steganographic or non-steganographic)} |
| Prompt 2 | ### Text: {input sentence}<br>### Question: Is the above text steganographic?<br>### Answer: {input label (yes or no)} |
| Prompt 3 | ### Here is a tweet: {input sentence}<br>### Question: Is the above tweet steganographic?<br>### Answer: {input label (yes or no)} |
| Prompt 4 | ### {input sentence}<br>### {input label (steganographic or non-steganographic)} |
| Prompt 5 | ### {input sentence}<br>### {input label (0 or 1)} |
| Prompt 6 | ### Text: {input sentence}<br>### Label: {input label (steganographic or non-steganographic)} |
| Prompt 7 | ### Text: {input sentence}<br>### Label: {input label (stega or cover)} |
| Prompt 8 | ### Text: {input sentence}<br>### Label: {input label (0 or 1)} |

As an example, we tested the effect of these prompts in Tweet+AC dataset, and the results are shown in Table VI.

In the prompts containing a question (Prompt 1,2,3), the detection performance for the "Yes/No" labels is better than the performance for the "Steganographic/Non-steganographic" labels. The efficiency of Bloomz is further improved by incorporating the textual category information ("tweet") in the input prompt. However, there are some minor performance changes for Llama.

For the prompts with only separators (Prompts 4 and 5), the effectiveness of Prompt 4 for generating long words is similar to that of the full prompt. On the other hand, Prompt 5, which outputs either 0 or 1, completely destroys the steganalysis capability of Bloomz, while Llama is still able to maintain a robust result.

Regarding the results from prompts 6, 7, and 8, we can verify that the most suitable output for LLMs is to be lin-

TABLE VIII
GENERIC STEGANALYSIS RESULT. THE RIGHTMOST TWO COLUMNS REPRESENT THE RESULT OF THE BEST BASELINE METHOD TRAINED IN THESE DATASETS, WHILE OUR MODEL IS ONLY TESTED.

| Datasets & Algorithms | | GS-Llama | | | | | | Best Baseline | |
|---|---|---|---|---|---|---|---|---|---|
| | | TP | FN | FP | TN | Accuracy | F1 score | Accuracy | F1 score |
| Movie | AC | 935 | 65 | 53 | 947 | 94.10 | 94.06 | 68.17 | 67.92 |
| | HC | 839 | 161 | 53 | 947 | 89.30 | 88.69 | 79.05 | 80.23 |
| | ADG | 898 | 102 | 53 | 947 | 92.25 | 92.06 | 71.80 | 68.81 |
| | DISCOP | 761 | 238 | 53 | 947 | **85.40** | **83.90** | 76.15 | 75.84 |
| News | AC | 744 | 256 | 124 | 876 | **81.00** | **79.66** | 70.25 | 68.28 |
| | HC | 669 | 331 | 124 | 876 | 77.25 | 74.62 | 77.90 | 79.64 |
| | ADG | 757 | 243 | 124 | 876 | **81.65** | **80.49** | 70.00 | 67.62 |
| Tweet | AC | 770 | 230 | 217 | 783 | 77.65 | 77.50 | 66.02 | 64.24 |
| | HC | 818 | 182 | 217 | 783 | 80.05 | 80.39 | 71.30 | 73.28 |
| | ADG | 787 | 213 | 217 | 783 | 78.50 | 78.54 | 71.80 | 68.81 |
| Commonsense | AC | 710 | 290 | 225 | 775 | **74.25** | **73.39** | 60.30 | 59.57 |
| | HC | 762 | 238 | 225 | 775 | **76.85** | **76.70** | 67.40 | 65.50 |
| | ADG | 688 | 312 | 225 | 775 | **73.15** | **71.93** | 59.65 | 57.26 |
| Aclimdb | AC | 920 | 80 | 33 | 967 | **94.35** | **94.21** | 85.50 | 85.50 |
| | HC | 978 | 22 | 33 | 967 | **97.25** | **97.30** | 92.25 | 92.23 |
| | ADG | 986 | 14 | 33 | 967 | **97.65** | **97.78** | 78.75 | 78.14 |

guistically precise, whereas meaning of "cover" is excessively broad, resulting in model outputs that deviate from our expectations. The difference in detection performance between numeric and long words is due to the model's understanding of steganographic/non-steganographic concepts. Owing to the resemblance of "Steganographic" and "Non-steganographic", the model produces lower losses for this output. Nevertheless, the use of these low-loss labels does not result in optimal detection performance.

To summarise, we anticipate that all inputs and outputs closely emulate natural language. Additionally, these inputs should be rational and unambiguous for LLMs to comprehend easily. Ideally, the output labels should display considerable variation at the token level, have a unique semantic interpretation and conflicting meanings.

### *E. Towards General Steganalysis*

After the sufficient experiments presented above, we found that there are 2 main challenges for LLMs in the steganalysis task. The first challenge is that short texts are more difficult to detect, which is also quite problematic for humans and baseline methods. The second challenge is that as the degree of text diversity increases, the decline in detection accuracy is difficult to compensate for.

With sufficient experimental preparation, now it's time to train a generic steganalysis LLM. First, in terms of model training settings, we choose Llama-7B training for 3 epochs with Prompt 2. Then for the training dataset, we extract 10000 sentences from Movie-AC and 5000 sentences from Tweet-HC as steganographic dataset, along with 10000 sentences from Movie-Natural and 5000 sentences from Tweet-Natural as natural dataset. From the results of domain-agnostic task it can be observed that the Movie dataset contains various texts and the distribution of this dataset is the most spread out, which enables our model to learn a diverse range of textual features. Thus we choose the Movie dataset as the basis. To enhance the models' ability to detect short text, we chose a part of the Tweet dataset as supplementary. Under these settings, our model took 9 hours to train on one RTX3090 GPU.

Besides the Movie and Tweet datasets, we test our model in new datasets (Aclimdb and Commonsense) and a new steganographic algorithm (DISCOP). Since we do not use any data from the News dataset, the results on the News dataset also demonstrate the capability of general detection of our model. The main results of general steganalysis task are shown in Table VIII.

In most cases, the detection rate and F1-score of our model are much higher than the baseline method that was trained and tested in the same dataset, with an increase in detection accuracy by approximately 10%. Compared with the results of domain-agnostic steganalysis that only uses the Movie-AC dataset, our model has significant gains. In this experiment, some different patterns are exposed. In the datasets that do not participate in training, the incorrectly detected samples are explicitly less than non-detected samples, which confirms our conjectures about over-fitting. We observed that in these datasets the incorrectly detected samples are shorter and with higher PPL, compared with the non-detected samples. Therefore, for the generality of the model, constructing a diverse dataset is significant for model training.

We will continue to improve this general steganalysis model, aiming at training a practical LLM for steganalysis.

## V. CONCLUSION

In this paper, we proposed a novel steganalysis method that different from the current mainstream classification-based methods. To the best of our knowledge, we are the first to construct a generation-based steganalysis method with LLMs. To directly generate the readable detection output without any additional deep learning module, we fine-tuned LLMs with LoRA framework, maintaining the original capability of LLMs. Our method showed superior performance in different scenarios compared to all existing steganalysis methods. Results show that the detection ability of our method is based on the fluency and rationality of the text, only a minor proportion of short stegos with low PPL are likely to escape the detection. Furthermore, our models demonstrated exceptional

capability in domain-agnostic tasks, providing robust detection results even when trained and tested on different datasets. This infers that LLMs gained wider-ranging characteristics of steganographic text during instruction fine-tuning, which remain effective despite significant variations in the text.

With the above results, we finally fine-tuned a general steganalysis model that can be applied to various tasks. In most cases, our method gains higher detection accuracy when only tested in the dataset than baseline methods trained in the same dataset.

We hope that our method will provide new insights for steganalysis in the era of LLMs. Our goal is to further improve the detection capabilities of our general steganalysis model and make it applicable to real-world situations.

## REFERENCES


[1] R. Cogranne, Q. Giboulot, and P. Bas, "Efficient steganography in jpeg images by minimizing performance of optimal detector," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1328–1343, 2022.

[2] M. Sharifzadeh, M. Aloraini, and D. Schonfeld, "Adaptive batch size image merging steganography and quantized gaussian image steganography," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 867–879, 2020.

[3] J. Butora and P. Bas, "Side-informed steganography for jpeg images by modeling decompressed images," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2683–2695, 2023.

[4] X. Yi, K. Yang, X. Zhao, Y. Wang, and H. Yu, "Ahcm: Adaptive huffman code mapping for audio steganography based on psychoacoustic model," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 8, pp. 2217–2231, 2019.

[5] J. Wu, B. Chen, W. Luo, and Y. Fang, "Audio steganography based on iterative adversarial attacks against convolutional neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2282–2294, 2020.

[6] Z.-L. Yang, X.-Q. Guo, Z.-M. Chen, Y.-F. Huang, and Y.-J. Zhang, "Rnn-stega: Linguistic steganography based on recurrent neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 5, pp. 1280–1295, 2018.

[7] Z.-L. Yang, S.-Y. Zhang, Y.-T. Hu, Z.-W. Hu, and Y.-F. Huang, "Vae-stega: linguistic steganography based on variational auto-encoder," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 880–895, 2020.

[8] S. Zhang, Z. Yang, J. Yang, and Y. Huang, "Provably secure generative linguistic steganography," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 3046–3055.

[9] C. S. de Witt, S. Sokota, J. Z. Kolter, J. N. Foerster, and M. Strohmeier, "Perfectly secure steganography using minimum entropy coupling," in *The Eleventh International Conference on Learning Representations*, 2022.

[10] J. Ding, K. Chen, Y. Wang, N. Zhao, W. Zhang, and N. Yu, "Discop: Provably secure steganography in practice based on "distribution copies"," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2023, pp. 2238–2255.

[11] T. Yang, H. Wu, B. Yi, G. Feng, and X. Zhang, "Semantic-preserving linguistic steganography by pivot translation and semantic-aware bins coding," *IEEE Transactions on Dependable and Secure Computing*, 2023.

[12] T. Fang, M. Jaggi, and K. Argyraki, "Generating steganographic text with lstms," in *Proceedings of ACL 2017, Student Research Workshop*, 2017, pp. 100–106.

[13] Z. M. Ziegler, Y. Deng, and A. M. Rush, "Neural linguistic steganography," *arXiv preprint arXiv:1909.01496*, 2019.

[14] J. Shen, H. Ji, and J. Han, "Near-imperceptible neural linguistic steganography via self-adjusting arithmetic coding," *arXiv preprint arXiv:2010.00677*, 2020.

[15] F. Z. Dai and Z. Cai, "Towards near-imperceptible steganographic text," *arXiv preprint arXiv:1907.06679*, 2019.

[16] Z. Yang, S. Jin, Y. Huang, Y. Zhang, and H. Li, "Automatically generate steganographic text based on markov model and huffman coding," *arXiv preprint arXiv:1811.04720*, 2018.

[17] E. Zielińska, W. Mazurczyk, and K. Szczypiorski, "Trends in steganography," *Communications of the ACM*, vol. 57, no. 3, pp. 86–95, 2014.

[18] W. Mazurczyk and S. Wendzel, "Information hiding: challenges for forensic experts," *Communications of the ACM*, vol. 61, no. 1, pp. 86–94, 2017.

[19] J. Kelley, "Terror groups hide behind web encryption." *USA Today (May 2001)*, 2001.

[20] D. Sieberg, "Bin laden exploits technology to suit his needs." *CNN (Sept. 2001)*, 2001.

[21] L. M. CAMERON, "With cryptography easier to detect, cybercriminals now hide malware in plain sight. call it steganography. here's how it works." *IEEE Computer Society*, 2018.

[22] Z. Chen, L. Huang, H. Miao, W. Yang, and P. Meng, "Steganalysis against substitution-based linguistic steganography based on context clusters," *Computers & Electrical Engineering*, vol. 37, no. 6, pp. 1071–1081, 2011.

[23] L. Xiang, X. Sun, G. Luo, and B. Xia, "Linguistic steganalysis using the features derived from synonym frequency," *Multimedia tools and applications*, vol. 71, pp. 1893–1911, 2014.

[24] Z. Yang, Y. Huang, and Y.-J. Zhang, "A fast and efficient text steganalysis method," *IEEE Signal Processing Letters*, vol. 26, no. 4, pp. 627–631, 2019.

[25] Z. Yang, Y. Huang, and Y. Zhang, "Ts-csw: Text steganalysis and hidden capacity estimation based on convolutional sliding windows," *Multimedia Tools and Applications*, vol. 79, pp. 18 293–18 316, 2020.

[26] Z. Yang, K. Wang, J. Li, Y. Huang, and Y.-J. Zhang, "Ts-rnn: text steganalysis based on recurrent neural net-

works," *IEEE Signal Processing Letters*, vol. 26, no. 12, pp. 1743–1747, 2019.

[27] H. Wu, B. Yi, F. Ding, G. Feng, and X. Zhang, "Linguistic steganalysis with graph neural networks," *IEEE Signal Processing Letters*, vol. 28, pp. 558–562, 2021.

[28] S. Li, J. Wang, and P. Liu, "Detection of generative linguistic steganography based on explicit and latent text word relation mining using deep learning," *IEEE Transactions on Dependable and Secure Computing*, 2022.

[29] S. Guo, J. Liu, Z. Yang, W. You, and R. Zhang, "Linguistic steganalysis merging semantic and statistical features," *IEEE Signal Processing Letters*, vol. 29, pp. 2128–2132, 2022.

[30] Y. Xue, L. Kong, W. Peng, P. Zhong, and J. Wen, "An effective linguistic steganalysis framework based on hierarchical mutual learning," *Information Sciences*, vol. 586, pp. 140–154, 2022.

[31] J. Wen, L. Gao, G. Fan, Z. Zhang, J. Jia, and Y. Xue, "Scl-stega: Exploring advanced objective in linguistic steganalysis using contrastive learning," in *Proceedings of the 2023 ACM Workshop on Information Hiding and Multimedia Security*, 2023, pp. 97–102.

[32] W. Peng, S. Li, Z. Qian, and X. Zhang, "Text steganalysis based on hierarchical supervised learning and dual attention mechanism," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[33] Q. Xu, R. Zhang, and J. Liu, "Linguistic steganalysis by enhancing and integrating local and global features," *IEEE Signal Processing Letters*, vol. 30, pp. 16–20, 2023.

[34] J. Zou, Z. Yang, S. Zhang, S. u. Rehman, and Y. Huang, "High-performance linguistic steganalysis, capacity estimation and steganographic positioning," in *International Workshop on Digital Watermarking*. Springer, 2020, pp. 80–93.

[35] J. Yang, Z. Yang, X. Ge, J. Zou, Y. Gao, and Y. Huang, "Link: Linguistic steganalysis framework with external knowledge," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[36] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, jan 2023.

[37] Y. Chen, R. Wang, H. Jiang, S. Shi, and R. Xu, "Exploring the use of large language models for reference-free text quality evaluation: A preliminary empirical study," *arXiv preprint arXiv:2304.00723*, 2023.

[38] M. B. et al., "Exploration of the effectiveness and characteristics of chatgpt in steganalysis tasks," 2023.

[39] Y. Niu, J. Wen, P. Zhong, and Y. Xue, "A hybrid r-bilstm-c neural network based text steganalysis," *IEEE Signal Processing Letters*, vol. 26, no. 12, pp. 1907–1911, 2019.

[40] H. Yang, Y. Bao, Z. Yang, S. Liu, Y. Huang, and S. Jiao, "Linguistic steganalysis via densely connected lstm with feature pyramid," in *Proceedings of the 2020 ACM Workshop on Information Hiding and Multimedia Security*, 2020, pp. 5–10.

[41] J. Zhang, J. Shen, L. Wang, and H. Lin, "Coverless text information hiding method based on the word rank map," in *Cloud Computing and Security*, X. Sun, A. Liu, H.-C. Chao, and E. Bertino, Eds. Cham: Springer International Publishing, 2016, pp. 145–155.

[42] J. Zhang, H. Huang, L. Wang, H. Lin, and D. Gao, "Coverless text information hiding method using the frequent words hash." *Int. J. Netw. Secur.*, vol. 19, no. 6, pp. 1016–1023, 2017.

[43] R. Bergmair, "Towards linguistic steganography: A systematic investigation of approaches, systems, and issues," *Final year thesis, B. Sc.(Hons.) in Computer Studies, The University of Derby*, 2004.

[44] M. J. Atallah, V. Raskin, M. Crogan, C. Hempelmann, F. Kerschbaum, D. Mohamed, and S. Naik, "Natural language watermarking: Design, analysis, and a proof-of-concept implementation," in *Information Hiding: 4th International Workshop, IH 2001 Pittsburgh, PA, USA, April 25–27, 2001 Proceedings 4*. Springer, 2001, pp. 185–200.

[45] T. Brown and e. a. Mann, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[46] A. Chowdhery and e. a. Narang, "Palm: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[47] B. Workshop, "Bloom: A 176b-parameter open-access multilingual language model," 2023.

[48] H. Touvron and T. L. et al., "Llama: Open and efficient foundation language models," 2023.

[49] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[50] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," 2023.

[51] W. Chiang and Z. e. a. Li, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, mar. 2023."

[52] OpenAI, "Gpt-4 technical report," 2023.

[53] E. J. Hu and Y. S. et al., "Lora: Low-rank adaptation of large language models," 2021.

[54] N. Muennighoff and T. W. et al., "Crosslingual generalization through multitask finetuning," 2023.